FLUTTER STATE MANAGEMENT

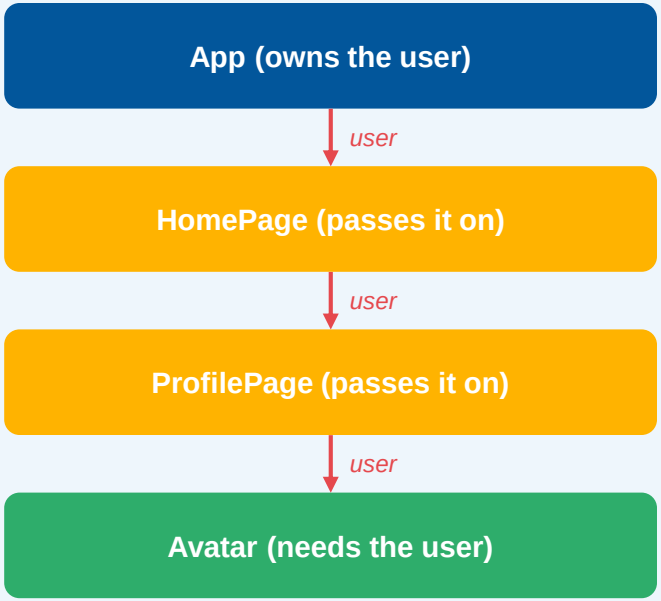
Production State Management: Riverpod

Declarative state reactivity with Riverpod 2.x/3.x: Providers, Notifiers, AsyncValue, and compile-time code generation

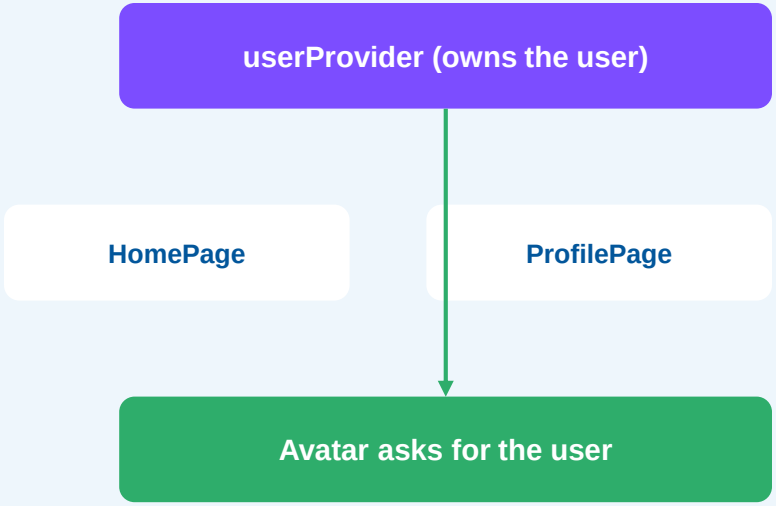
START HERE

The problem: sharing data between widgets

WITHOUT PROVIDERS



WITH PROVIDERS



Pages in the middle are left alone. Avatar reaches the data directly.

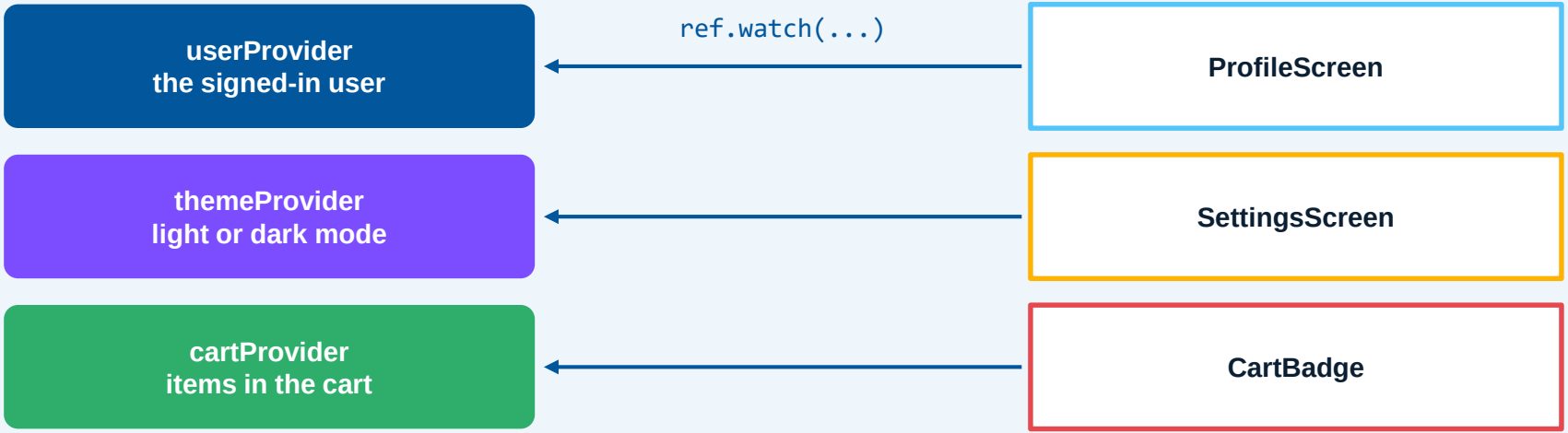
Passing data through every layer is slow to write and easy to break. Providers remove the middlemen.

Sources: [3] Getting started, [4] Providers

START HERE

What is a provider?

A provider is a named box that holds one piece of state and knows how to create it.



- Any widget can ask for a provider by name
- When the state changes, the widgets that watch it update themselves

Sources: [4] Providers, [6] Refs

START HERE

What is Riverpod?

1 Creates

Builds each provider the first time something asks for it.

2 Shares

Gives every widget the same value instead of copies.

3 Updates

Tells widgets to redraw when the state changes.

4 Cleans up

Throws away state nobody uses and closes resources.

Riverpod is a library that manages providers for you.

You describe what the state is. Riverpod handles when to build it, who gets it and when to rebuild the screen.

Sources: [3] Getting started, [4] Providers

START HERE

Riverpod in three steps

- 1. Declare: describe the state once, in a provider
- 2. Watch: widgets read it with ref.watch and update automatically
- 3. Change: call a method with ref.read when something happens
- That is the whole idea; the rest of the deck is detail

counter_app.dart

```dart
// 1. Declare
class Counter extends Notifier<int> {
  @override
  int build() => 0;            // start value
  void increment() => state++; // how it changes
}
final counterProvider =
    NotifierProvider<Counter, int>(Counter.new);

// 2. Watch
class CounterText extends ConsumerWidget {
  const CounterText({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Text('$count');
  }
}

// 3. Change
onPressed: () =>
    ref.read(counterProvider.notifier).increment(),
```

Sources: [3] Getting started, [4] Providers, [5] Consumers

START HERE

Key words in plain English

P Provider

A labeled box that holds one piece of state.

R ref

Your way to reach providers from code.

C ConsumerWidget

A widget that is allowed to use ref.

N Notifier

A provider with buttons (methods) that change its state.

A AsyncValue

A result that is loading, has data, or failed.

S ProviderScope

The place at the top of the app where all boxes live.

Sources: [4] Providers, [5] Consumers, [14] AsyncValue API

OVERVIEW

Agenda

1 Foundations

Why Riverpod, packages, setup, the provider graph

2 Core concepts

Provider types, Ref, consumers, scopes, Notifiers

3 Async with AsyncValue

Futures, streams, loading and error states, family, disposal

4 Riverpod 3.x

Retry, pause/resume, Ref.mounted, mutations, offline cache

5 Code generation

@riverpod, generated providers, build_runner, lint

6 Why each API matters

Purpose, benefits and examples for every provider type and Ref function

Sources: [3] Getting started

Why production state management?

1 Predictable

One source of truth per piece of state and a clear data flow from providers to widgets.

2 Testable

Business logic lives outside widgets, and any dependency can be overridden in tests.

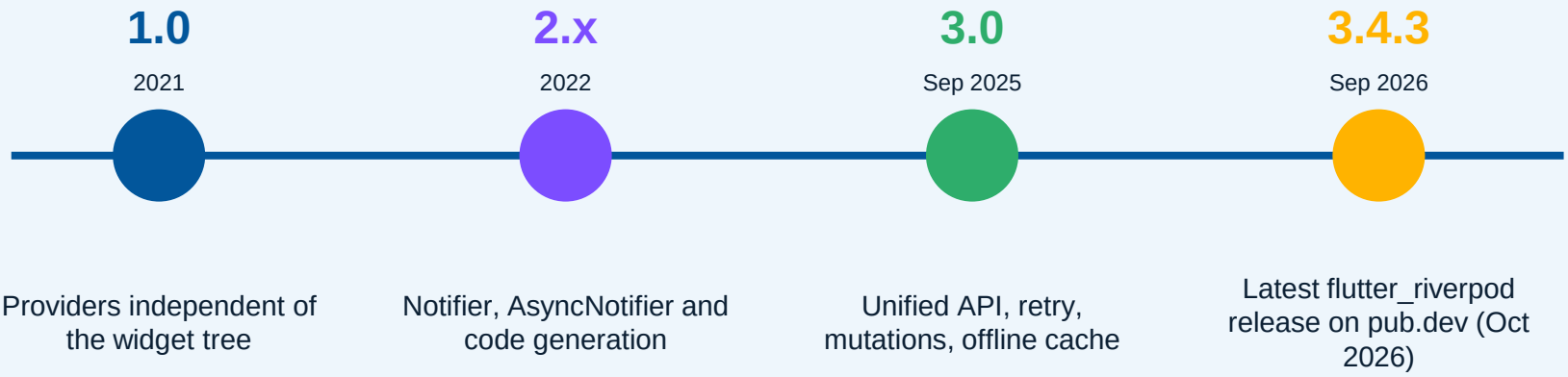
3 Scalable

Providers form a dependency graph that grows feature by feature without tangling widgets.

UI = f(state). Providers describe how state is created and derived; widgets only watch it and react.

Sources: [3] Getting started, [19] Riverpod for Provider users

Riverpod across versions



- Riverpod 3.0 is a transition toward a simpler, unified API with a few lifecycle changes, so upgrade carefully
- Two features, mutations and offline persistence, are still experimental

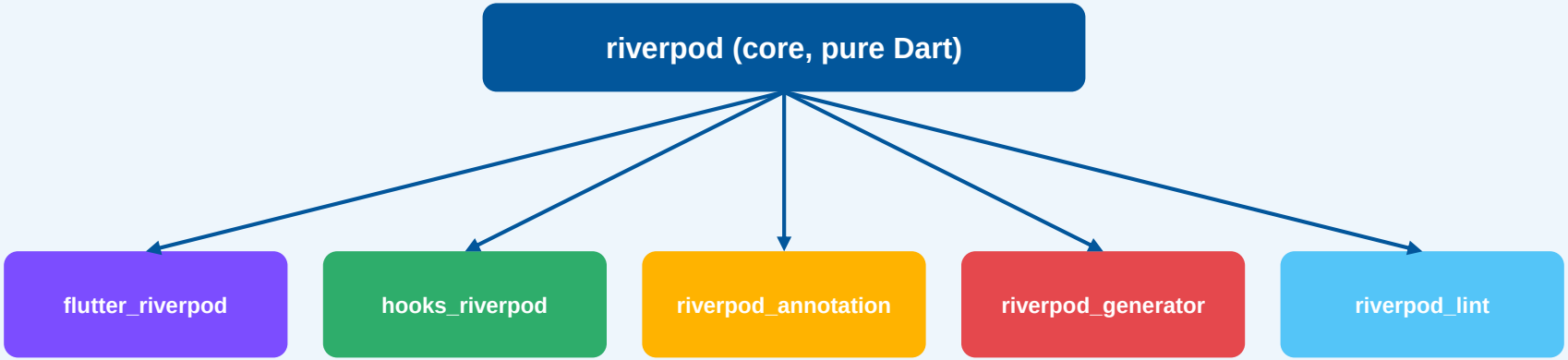
Sources: [1] What's new in 3.0, [14] pub.dev API docs, [20] GitHub

Riverpod vs the provider package

provider package	Riverpod
• Looks providers up through BuildContext	• Providers are top-level declarations
• A missing provider fails at runtime	• References are checked at compile time
• Async data needs manual loading and error handling	• AsyncValue models loading, data and error
• Overrides tied to the widget tree	• Easy overrides for tests and scopes

Sources: [19] Riverpod for Provider users, [4] Providers

The Riverpod package family



Optional add-ons: riverpod_sqflite provides storage for the experimental offline persistence feature.

Sources: [21] pub.dev packages, [12] About code generation, [10] Offline persistence

Install and set up

- Add flutter_riverpod; codegen packages are optional
- flutter pub add resolves compatible latest versions
- Wrap the app in a single ProviderScope at the root
- ProviderScope stores the state of every provider

terminal + main.dart

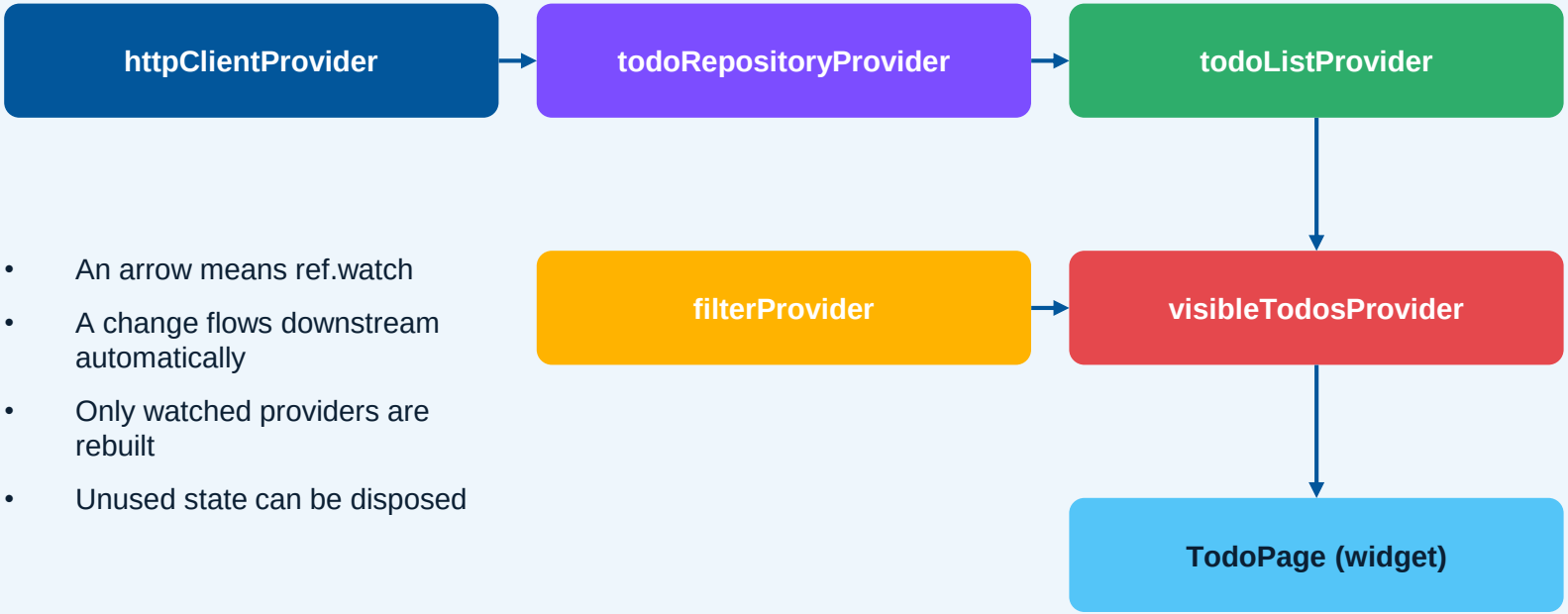
```
# Terminal (latest: flutter_riverpod ^3.4.3)
flutter pub add flutter_riverpod
flutter pub add riverpod_annotation
flutter pub add dev:riverpod_generator
flutter pub add dev:build_runner
flutter pub add dev:riverpod_lint

// main.dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

void main() {
  runApp(const ProviderScope(child: MyApp()));
}
```

Sources: [3] Getting started, [21] pub.dev

The provider graph



- An arrow means ref.watch
- A change flows downstream automatically
- Only watched providers are rebuilt
- Unused state can be disposed

Sources: [4] Providers, [6] Refs

Provider types at a glance

P Provider

Synchronous value: configuration, computed data, repositories.

F FutureProvider

One asynchronous result exposed as AsyncValue.

S StreamProvider

Stream events exposed as AsyncValue.

N NotifierProvider

Synchronous state plus methods that change it.

A AsyncNotifierProvider

Asynchronous state plus methods. The feature workhorse.

SN StreamNotifierProvider

Stream-backed state with methods.

Sources: [4] Providers

Provider: values and dependency injection

- Provider exposes a value that never changes by itself
- Rebuilds when a watched provider changes
- Use ref.onDispose to release resources
- Great for repositories, clients and config

providers.dart

```dart
final apiBaseUrlProvider = Provider<String>((ref) {
  return 'https://api.example.com';
});

final httpClientProvider = Provider<Dio>((ref) {
  final dio = Dio(BaseOptions(
    baseUrl: ref.watch(apiBaseUrlProvider),
  ));
  ref.onDispose(dio.close);
  return dio;
});

final todoRepositoryProvider =
    Provider<TodoRepository>((ref) {
  return HttpTodoRepository(
    ref.watch(httpClientProvider),
  );
});
```

Sources: [4] Providers

Ref: watch, read and listen

W ref.watch

Reads and subscribes. The caller rebuilds when the value changes.

R ref.read

Reads once without subscribing. Use inside callbacks.

L ref.listen

Runs a callback on change. Use for side effects.

ref_usage.dart

```dart
final repoProvider = Provider<Repo>((ref) {
  final url = ref.watch(apiBaseUrlProvider);       // rebuilds this provider on change
  ref.listen(authProvider, (prev, next) {          // side effect, no rebuild
    if (next == null) ref.invalidateSelf();
  });
  ref.onDispose(() => debugPrint('repo disposed'));
  return Repo(url);
});

// In a widget callback only:
onPressed: () => ref.read(counterProvider.notifier).increment(),
```

Sources: [6] Refs, [18] DO/DON'T

Consumers: getting a WidgetRef

- ConsumerWidget is the default choice
- Consumer rebuilds only a subtree
- ConsumerStatefulWidget when you need State lifecycle
- ref.read is fine in initState; ref.watch is not

consumers.dart

```dart
// 1. ConsumerWidget
class Home extends ConsumerWidget {
  const Home({super.key});
  @override
  Widget build(BuildContext c, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Text('$count');
  }
}

// 2. Consumer: rebuild a subtree only
Consumer(builder: (context, ref, child) {
  return Text('${ref.watch(counterProvider)}');
})

// 3. ConsumerStatefulWidget
class _FormState extends ConsumerState<Form> {
  @override
  void initState() {
    super.initState();
    ref.read(analyticsProvider).logOpen();
  }
}
```

Sources: [5] Consumers

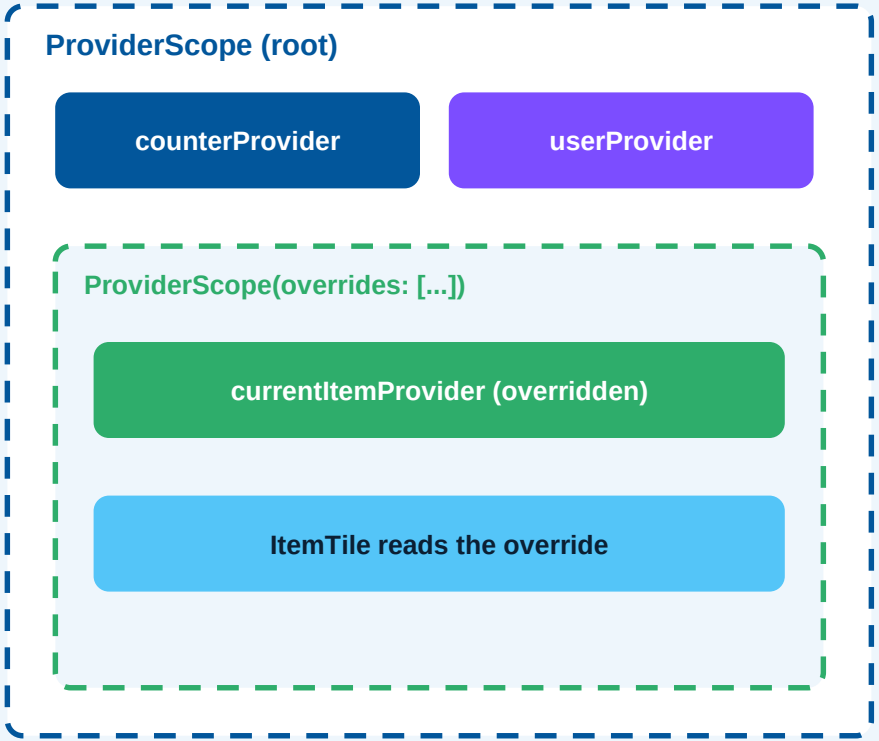
WidgetRef do and don't

Do	Don't
• ref.watch in build() for displayed values	• ref.read in build() for values you display
• ref.read in onPressed and other callbacks	• ref.watch inside callbacks
• ref.listen for snackbars, dialogs, navigation	• Side effects directly inside build()
• select() to rebuild on a slice only	• Use ref after the widget is disposed

Sources: [5] Consumers, [18] DO/DON'T

CORE CONCEPTS

ProviderScope and ProviderContainer



scopes.dart

```dart
void main() {
  runApp(const ProviderScope(child: MyApp()));
}

// Override in a subtree
ProviderScope(
  overrides: [
    currentItemProvider.overrideWithValue(item),
  ],
  child: const ItemTile(),
)

// Pure Dart and tests: no widgets needed
final container = ProviderContainer();
final value = container.read(counterProvider);
container.dispose();
```

Sources: [22] Containers, [16] Scoping, [13] Testing

NotifierProvider

- state is the only mutable field and is replaced, not edited
- Public methods form the API of the feature
- build() returns the initial state and may watch other providers
- Assigning a new value notifies listeners

counter.dart

```dart
class Counter extends Notifier<int> {
  @override
  int build() => 0; // initial state

  void increment() => state++;
  void add(int n) => state += n;
  void reset() => state = 0;
}

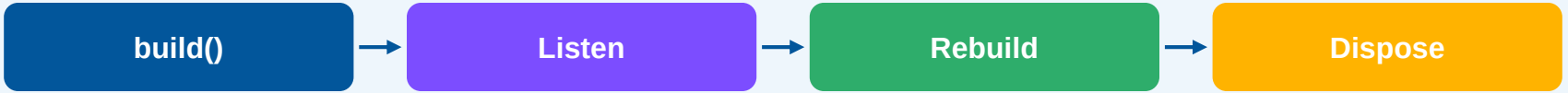
final counterProvider =
    NotifierProvider<Counter, int>(Counter.new);

// In a widget
final count = ref.watch(counterProvider);
ref.read(counterProvider.notifier).increment();
```

Sources: [4] Providers, [1] What's new in 3.0

CORE CONCEPTS

Notifier lifecycle



Creates the initial state

Widgets and providers subscribe

A watched provider changes; build() runs again

Container ends, or auto-dispose with no listeners

search_notifier.dart

```dart
class Search extends Notifier<List<Item>> {
  @override
  List<Item> build() {
    final query = ref.watch(queryProvider);   // rebuild when the query changes
    ref.onDispose(() => debugPrint('cleanup')); // runs before the next build or disposal
    return _filter(ref.watch(itemsProvider), query);
  }
  // In 3.0 a new Notifier instance is created on every rebuild,
  // so do not keep long-lived mutable fields on it.
}
```

Sources: [2] Migration guide, [7] Automatic disposal

Immutable state and collections

- Create a new object or list on every change
- Providers compare states with ==, so use value equality
- Records, freezed or equatable all work
- Never mutate a list inside state

todo_list.dart

```dart
@immutable
class Todo {
  const Todo({
    required this.id,
    required this.title,
    this.done = false,
  });
  final String id;
  final String title;
  final bool done;
  Todo copyWith({String? title, bool? done}) => Todo(
      id: id, title: title ?? this.title,
      done: done ?? this.done);
}

class TodoList extends Notifier<List<Todo>> {
  @override
  List<Todo> build() => const [];
  void add(Todo t) => state = [...state, t];
  void remove(String id) =>
      state = [for (final t in state) if (t.id != id) t];
}
```

Sources: [1] What's new in 3.0, [18] DO/DON'T

FutureProvider

- Runs the future once and exposes an AsyncValue
- Cached until invalidated or disposed
- Watching other providers re-runs it automatically
- Use switch with AsyncData, AsyncError, AsyncLoading

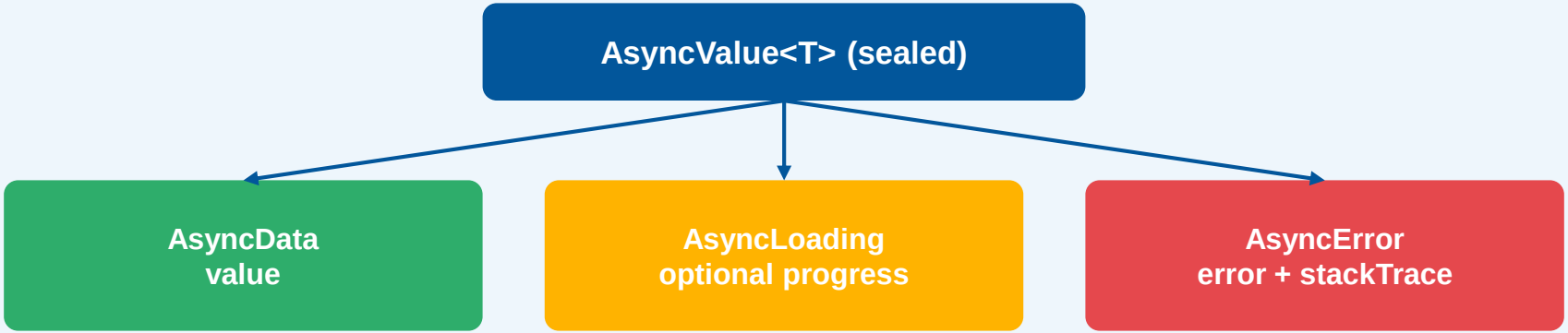
user_provider.dart

```dart
final userProvider = FutureProvider<User>((ref) async {
  final repo = ref.watch(userRepositoryProvider);
  return repo.fetchCurrentUser();
});

// In a widget
final user = ref.watch(userProvider); // AsyncValue<User>
return switch (user) {
  AsyncData(:final value) => Text(value.name),
  AsyncError(:final error) => Text('Error: $error'),
  _ => const CircularProgressIndicator(),
};
```

Sources: [4] Providers, [14] AsyncValue API

AsyncValue anatomy



Read

- value (nullable), error, stackTrace
- hasValue, hasError
- requireValue (throws if no value)

Flags

- isLoading, isRefreshing, isReloading
- retrying (automatic retry in progress)
- isFromCache (offline persistence)

Sources: [14] AsyncValue API reference

Rendering AsyncValue

- The sealed class gives exhaustive switches
- value is nullable and handy for cached data
- when() skips loading on refresh by default
- requireValue throws when there is no value

async_ui.dart

```dart
// Pattern matching (exhaustive)
return switch (todos) {
  AsyncData(:final value) => TodoListView(value),
  AsyncError(:final error) => ErrorView('$error'),
  AsyncLoading() => const Center(
      child: CircularProgressIndicator()),
};

// Callbacks
todos.when(
  data: (items) => TodoListView(items),
  error: (e, st) => ErrorView('$e'),
  loading: () => const CircularProgressIndicator(),
);

// Reuse the last value
final items = todos.value ?? const [];
```

Sources: [14] AsyncValue API reference, [1] What's new in 3.0

AsyncNotifier

- build() returns a Future and sets the initial state
- Methods update state with AsyncLoading, AsyncData or guard
- AsyncValue.guard turns exceptions into AsyncError
- Use ref.read for one-off dependencies in methods

todo_list.dart

```dart
class TodoList extends AsyncNotifier<List<Todo>> {
  @override
  Future<List<Todo>> build() {
    final repo = ref.watch(todoRepositoryProvider);
    return repo.fetchAll();
  }

  Future<void> add(String title) async {
    final repo = ref.read(todoRepositoryProvider);
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      await repo.create(title);
      return repo.fetchAll();
    });
  }
}

final todoListProvider =
    AsyncNotifierProvider<TodoList, List<Todo>>(
        TodoList.new);
```

Sources: [4] Providers, [14] AsyncValue API

Optimistic updates and refresh states

- Update the UI first, then confirm with the server
- Roll back on failure and rethrow
- isRefreshing: forced recompute after a value existed
- isReloading: recompute after a dependency changed

optimistic.dart

```dart
Future<void> toggle(String id) async {
  final previous = await future;
  state = AsyncData([
    for (final t in previous)
      if (t.id == id) t.copyWith(done: !t.done) else t,
  ]);
  try {
    await ref.read(todoRepositoryProvider).toggle(id);
  } catch (e, st) {
    if (!ref.mounted) return;
    state = AsyncData(previous); // rollback
    Error.throwWithStackTrace(e, st);
  }
}
```

Sources: [14] AsyncValue API, [1] What's new in 3.0

StreamProvider and StreamNotifier

- Each event becomes a new AsyncValue
- Re-subscribes when watched providers change
- In 3.0 stream events are filtered with ==
- Override updateShouldNotify to customize

streams.dart

```dart
final messagesProvider =
    StreamProvider<List<Message>>((ref) {
  final chatId = ref.watch(chatIdProvider);
  return ref.watch(chatRepositoryProvider).watch(chatId);
});

class Ticker extends StreamNotifier<int> {
  @override
  Stream<int> build() => Stream.periodic(
      const Duration(seconds: 1), (i) => i);
}

final tickerProvider =
    StreamNotifierProvider<Ticker, int>(Ticker.new);

// UI
ref.watch(messagesProvider).when(
  data: (m) => MessageList(m),
  error: (e, _) => Text('$e'),
  loading: () => const LinearProgressIndicator(),
);
```

Sources: [4] Providers, [1] What's new in 3.0

Family: providers with parameters

- One cached state per argument
- Arguments need proper == and hashCode
- 3.x Notifiers take arguments through the constructor
- Combine with autoDispose for screen-scoped data

family.dart

```dart
final userProvider =
    FutureProvider.family<User, String>((ref, id) {
  return ref.watch(userRepositoryProvider).fetch(id);
});

// One cached state per argument
ref.watch(userProvider('42'));
ref.watch(userProvider('7'));

// Notifier family in 3.x
class ItemCounter extends Notifier<int> {
  ItemCounter(this.itemId);
  final String itemId;
  @override
  int build() => 0;
}
final itemCounterProvider =
    NotifierProvider.family<ItemCounter, int, String>(
        ItemCounter.new);
```

Sources: [8] Family, [2] Migration guide

autoDispose and keepAlive



A widget or provider watches it

No listeners remain

State is discarded; rebuilt on next use

cache_policy.dart

```dart
final searchProvider =
    FutureProvider.autoDispose<List<Item>>((ref) async {
  final link = ref.keepAlive();   // keep the state alive until link.close()
  final timer = Timer(const Duration(minutes: 5), link.close);
  ref.onDispose(timer.cancel);    // always release resources
  return ref.watch(apiProvider).search();
});

// Rule of thumb: screen data -> autoDispose, app-wide data -> keepAlive
```

Sources: [7] Automatic disposal

invalidate vs refresh

I ref.invalidate(p)

Marks the state stale. It recomputes when next needed, or at once if listened.

R ref.refresh(p)

Invalidates and returns the new value immediately. Await .future for async.

refresh.dart

```dart
// Pull to refresh
RefreshIndicator(
  onRefresh: () => ref.refresh(todoListProvider.future),
  child: const TodoListView(),
)

// Log out: reset dependent state
ref.invalidate(todoListProvider);
ref.invalidate(userProvider);
```

Sources: [6] Refs, [4] Providers

Rebuild less with select

- select watches one slice of a provider
- The widget rebuilds only when that slice changes
- selectAsync awaits one field of an async provider
- Works with ref.listen too

select.dart

```dart
// Rebuilds only when the name changes
final name = ref.watch(
  userProvider.select((user) => user.name),
);

// Inside another provider: await one field
final avatar = await ref.watch(
  profileProvider.selectAsync((p) => p.avatarUrl),
);

// Listen to a slice
ref.listen(
  cartProvider.select((c) => c.items.length),
  (prev, next) => debugPrint('items: $next'),
);
```

Sources: [17] Reduce rebuilds, [6] Refs

ref.listen for side effects

- Show snackbars, dialogs or navigate on state changes
- The callback gets previous and next values
- Do not trigger side effects inside build()
- Use listenManual in ConsumerState if needed

checkout_page.dart

```dart
class CheckoutPage extends ConsumerWidget {
  const CheckoutPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    ref.listen<AsyncValue<void>>(
      checkoutControllerProvider,
      (prev, next) {
        if (next.hasError && !next.isLoading) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('${next.error}')),
          );
        }
      },
    );
    return const CheckoutForm();
  }
}
```

Sources: [5] Consumers, [6] Refs

Derived providers

- Compute values from other providers with ref.watch
- The result is cached and only recomputed when inputs change
- Keep derived logic out of widgets
- Chain as many layers as you need

total_provider.dart

```dart
final cartProvider =
    NotifierProvider<Cart, List<Item>>(Cart.new);

final discountProvider = Provider<double>((ref) => 0.1);

final totalProvider = Provider<double>((ref) {
  final items = ref.watch(cartProvider);
  final discount = ref.watch(discountProvider);
  final sum = items.fold<double>(
      0, (s, i) => s + i.price);
  return sum * (1 - discount);
});

// Widget
Text(ref.watch(totalProvider).toStringAsFixed(2))
```

Sources: [4] Providers, [6] Refs

What's new in 3.0 and later

R Automatic retry

Failing providers refresh with exponential backoff.

M Mutations

Experimental. Lets the UI react to side-effects.

O Offline persistence

Experimental. Cache provider state in a database.

Ref Ref.mounted

Check if a provider is still usable after an await.

U Unified API

No AutoDispose or Family variants; a single Ref.

P Pause / resume

Listeners in off-screen widgets are paused.

Sources: [1] What's new in Riverpod 3.0, [2] Migration guide

Unified API: 2.x vs 3.x

Riverpod 2.x

```
class Counter
    extends AutoDisposeFamilyNotifier<int, String> {
  @override
  int build(String arg) => 0;
}

final counterProvider = NotifierProvider
    .autoDispose
    .family<Counter, int, String>(Counter.new);

// code generation
Example example(ExampleRef ref) {
  return Example();
}
```

Riverpod 3.x

```
class Counter extends Notifier<int> {
  Counter(this.arg);
  final String arg;

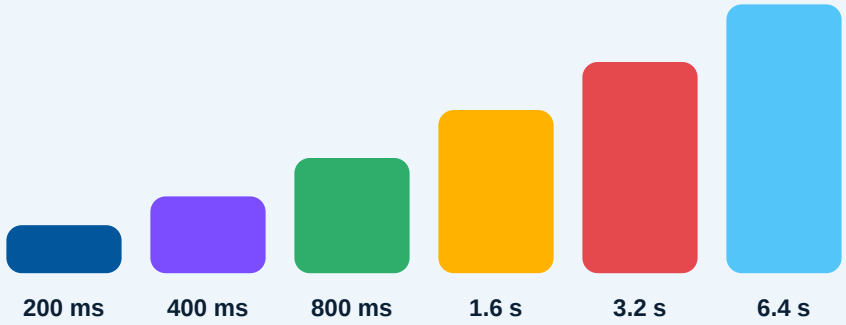
  @override
  int build() => 0;
}

final counterProvider = NotifierProvider
    .autoDispose
    .family<Counter, int, String>(Counter.new);

// code generation
Example example(Ref ref) {
  return Example();
}
```

Sources: [1] What's new in 3.0, [2] Migration guide

Automatic retry



Delay doubles after each failed attempt (bar heights illustrative)

- Default: retry any error until success or disposal
- Starts at 200 ms and doubles up to 6.4 s
- ProviderException errors are not retried by default

retry.dart

```dart
ProviderScope(
  retry: (retryCount, error) {
    if (error is UnauthorizedException) return null;
    if (retryCount > 5) return null; // give up
    return Duration(seconds: retryCount * 2);
  },
  child: const MyApp(),
)

// Disable for one provider
final todoListProvider =
    NotifierProvider<TodoList, List<Todo>>(
  TodoList.new,
  retry: (retryCount, error) => null,
);
```

Sources: [11] Automatic retry, [1] What's new in 3.0

Pause and resume

Home page
(not visible)
listeners paused

Settings page
(visible)
listeners active

- Riverpod uses TickerMode to detect visibility
- A provider used only by paused providers is paused too
- While rebuilding, old subscriptions stay paused until done

pause.dart

```dart
// Keep listening even when off-screen
TickerMode(
  enabled: true,
  child: Consumer(builder: (context, ref, _) {
    return Text('${ref.watch(socketProvider)}');
  }),
)

// Manual control of a listener
final sub = ref.listen(
  todoListProvider,
  (previous, next) {},
);
sub.pause();
sub.resume();
```

Sources: [1] What's new in 3.0, [2] Migration guide

Ref.mounted and disposal safety

- Using a disposed Ref now throws UnmountedRefException
- Guard code after an await with ref.mounted
- Or cancel pending work with ref.onDispose
- Works like BuildContext.mounted

mounted.dart

```dart
class TodoList extends Notifier<List<Todo>> {
  @override
  List<Todo> build() => const [];

  Future<void> add(String title) async {
    final todo = await api.create(title);
    // The provider may have been disposed or
    // rebuilt during the await.
    if (!ref.mounted) return;
    state = [...state, todo];
  }
}
```

Sources: [1] What's new in 3.0, [2] Migration guide

ProviderException

- Reading a failed provider throws a ProviderException
- It wraps the original error and its stack trace
- AsyncValue.error, ref.listen onError and observers still get the original error
- Check e.exception in try/catch blocks

errors.dart

```dart
try {
  await ref.read(userProvider.future);
} on ProviderException catch (e) {
  if (e.exception is NotFoundException) {
    showNotFound();
  }
}

// AsyncValue still exposes the original error
final user = ref.watch(userProvider);
if (user.error is NotFoundException) {
  // handle in the UI
}
```

Sources: [1] What's new in 3.0, [2] Migration guide

Update filtering with ==

- All providers now filter updates with ==
- Some listeners may fire more or less than before
- Give state classes value equality
- Override updateShouldNotify to opt out

equality.dart

```dart
@immutable
class Cart {
  const Cart(this.items);
  final List<String> items;

  @override
  bool operator ==(Object other) =>
      other is Cart && listEquals(items, other.items);

  @override
  int get hashCode => Object.hashAll(items);
}

class TodoStream extends StreamNotifier<Todo> {
  @override
  Stream<Todo> build() => repo.watchLatest();

  @override
  bool updateShouldNotify(
          AsyncValue<Todo> prev, AsyncValue<Todo> next) =>
      true;
}
```

Sources: [1] What's new in 3.0, [2] Migration guide

Legacy providers

StateProvider (legacy)	→ NotifierProvider
StateNotifierProvider (legacy)	→ NotifierProvider
ChangeNotifierProvider (legacy)	→ NotifierProvider + immutable state

before.dart

```dart
import 'package:flutter_riverpod/legacy.dart';

final counterProvider =
    StateProvider<int>((ref) => 0);

ref.read(counterProvider.notifier).state++;
```

after.dart

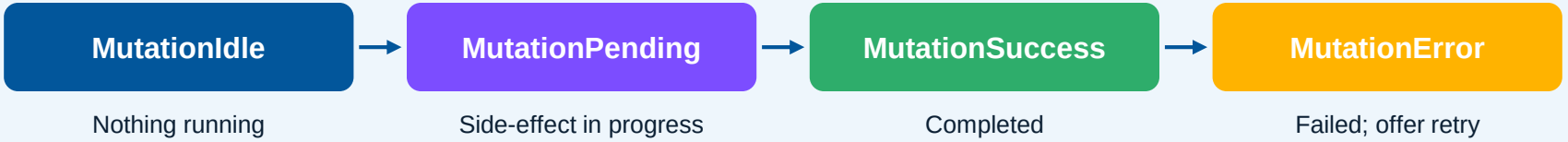
```dart
class Counter extends Notifier<int> {
  @override
  int build() => 0;
  void increment() => state++;
}

final counterProvider =
    NotifierProvider<Counter, int>(Counter.new);

ref.read(counterProvider.notifier).increment();
```

Sources: [2] Migration guide, [1] What's new in 3.0

Mutations (experimental)



add_todo_button.dart

```dart
// Experimental API: may change without a major version bump
final addTodoMutation = Mutation<void>();

// in ConsumerWidget.build
final addTodo = ref.watch(addTodoMutation);
return switch (addTodo) {
  MutationIdle() || MutationError() => FilledButton(
      onPressed: () => addTodoMutation.run(ref, () async {
        await ref.read(todoListProvider.notifier).add('New');
      }),
      child: const Text('Submit')),
  MutationPending() => const CircularProgressIndicator(),
  MutationSuccess() => const Text('Todo added!'),
};
```

Sources: [9] Mutations, [1] What's new in 3.0

Offline persistence (experimental)

- Opt in per Notifier with persist()
- riverpod_sqflite provides JsonSqFliteStorage
- Persisted value shows while the network loads
- AsyncValue.isFromCache tells cache from server

persist.dart

```dart
final storageProvider =
    FutureProvider<JsonSqFliteStorage>((ref) async {
  return JsonSqFliteStorage.open(
    join(await getDatabasesPath(), 'riverpod.db'),
  );
});

class TodosNotifier extends AsyncNotifier<List<Todo>> {
  @override
  FutureOr<List<Todo>> build() async {
    persist(
      ref.watch(storageProvider.future),
      key: 'todos',
      encode: jsonEncode,
      decode: (json) => _decodeTodos(json),
    );
    return fetchTodos(); // cache shows meanwhile
  }
}
```

Sources: [10] Offline persistence, [1] What's new in 3.0, [21] riverpod_sqflite

Why code generation?

1 Less boilerplate

One annotation replaces the provider declaration.

2 Plain parameters

Family arguments are ordinary function parameters.

3 Auto-dispose by default

Opt out with keepAlive: true.

4 Generics

Generated providers can define type parameters (3.0).

5 Safe scoping

riverpod_lint detects missing overrides.

6 Optional

Mix generated and hand-written providers freely.

Sources: [12] About code generation, [1] What's new in 3.0

Setting up the generator

- riverpod_annotation is a normal dependency
- riverpod_generator and build_runner are dev dependencies
- Add a part directive for every file
- watch regenerates on save

terminal + todos.dart

```
flutter pub add flutter_riverpod
flutter pub add riverpod_annotation
flutter pub add dev:riverpod_generator
flutter pub add dev:build_runner
flutter pub add dev:riverpod_lint

# generate once
dart run build_runner build -d

# or watch while coding
dart run build_runner watch -d

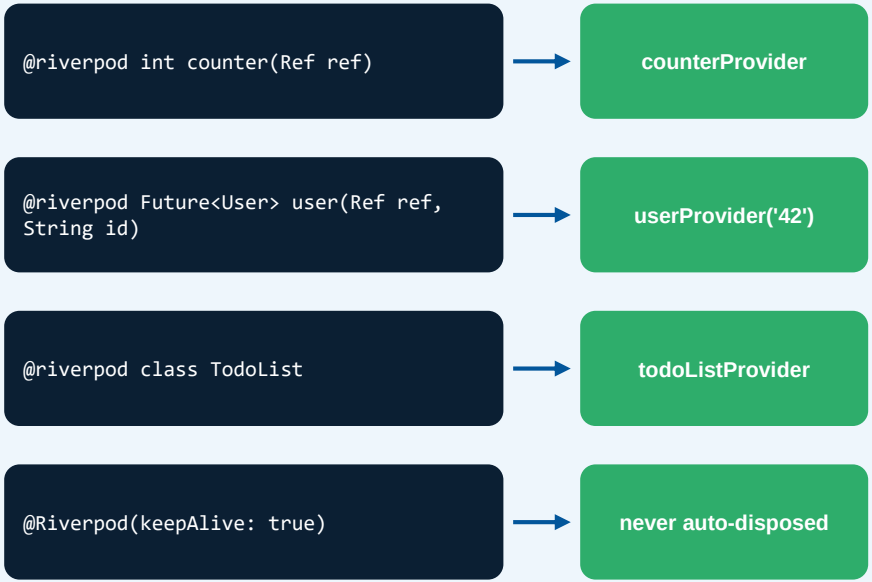
// todos.dart
import 'package:riverpod_annotation/riverpod_annotation.dart';

part 'todos.g.dart';
```

Sources: [12] About code generation

CODE GENERATION

@riverpod functions



providers.dart

```dart
part 'providers.g.dart';

@riverpod
String apiBaseUrl(Ref ref) => 'https://api.test';

@riverpod
Future<User> user(Ref ref, String id) async {
  final repo = ref.watch(userRepositoryProvider);
  return repo.fetch(id);
}

// Usage
ref.watch(apiBaseUrlProvider);
ref.watch(userProvider('42'));
```

Sources: [12] About code generation, [1] What's new in 3.0

CODE GENERATION

@riverpod classes

- Extend the generated _$ClassName base
- build() declares the type: sync, Future or Stream
- The provider name is the class name in lowerCamelCase plus Provider
- Same lifecycle and API as hand-written Notifiers

todo_list.dart

```dart
part 'todo_list.g.dart';

@riverpod
class TodoList extends _$TodoList {
  @override
  Future<List<Todo>> build() {
    final repo = ref.watch(todoRepositoryProvider);
    return repo.fetchAll();
  }

  Future<void> add(String title) async {
    final repo = ref.read(todoRepositoryProvider);
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      await repo.create(title);
      return repo.fetchAll();
    });
  }
}

ref.watch(todoListProvider);
ref.read(todoListProvider.notifier).add('Milk');
```

Sources: [12] About code generation

Parameters and generics

- Positional and named parameters become family arguments
- Named parameters stay named at the call site
- 3.0 adds generic providers
- Type arguments are passed when watching

family_gen.dart

```dart
@riverpod
Future<List<Post>> posts(
  Ref ref, {
  required String category,
  int page = 1,
}) {
  return ref.watch(postRepoProvider).list(category, page);
}

ref.watch(postsProvider(category: 'dart', page: 2));

// Generic provider (3.0)
@riverpod
T multiply<T extends num>(Ref ref, T a, T b) => a * b;

int i = ref.watch(multiplyProvider<int>(2, 3));
double d = ref.watch(multiplyProvider<double>(2.5, 3.5));
```

Sources: [1] What's new in 3.0, [8] Family

CODE GENERATION

keepAlive and scoped providers

- keepAlive: true keeps state for the whole session
- dependencies: [] marks a scoped provider
- @Dependencies declares who uses it
- Lint checks every override exists

scoped.dart

```dart
// Lives for the whole app session
@Riverpod(keepAlive: true)
AuthRepository authRepository(Ref ref) {
  return AuthRepository(ref.watch(httpClientProvider));
}

// Scoped: must be overridden below
@Riverpod(dependencies: [])
Future<int> currentItem(Ref ref) =>
    throw UnimplementedError();

// Declares the dependency so lint can verify it
@Dependencies([currentItem])
class ItemTile extends ConsumerWidget {
  const ItemTile({super.key});
  // ...
}
```

Sources: [1] What's new in 3.0, [16] Scoping providers

What the generator produces



Annotations and part directive

Runs riverpod_generator

Providers and _$ base classes

workflow.sh

```
dart run build_runner watch -d      # regenerate on save

# analysis_options.yaml: keep generated files out of analysis noise
analyzer:
  exclude:
    - '**/*.g.dart'

# Team choice: commit *.g.dart files, or ignore them and generate in CI.
# Whichever you pick, make it consistent across the repo.
```

Sources: [12] About code generation

CODE GENERATION

riverpod_lint

- Replaces the old AutoDispose compile errors with lint rules
- Detects missing overrides of scoped providers
- Run dart analyze in CI
- Follow the DO/DON'T guide for ref usage

lint_examples.dart

```
// DON'T: watching in a callback
onPressed: () {
  final n = ref.watch(counterProvider);
}

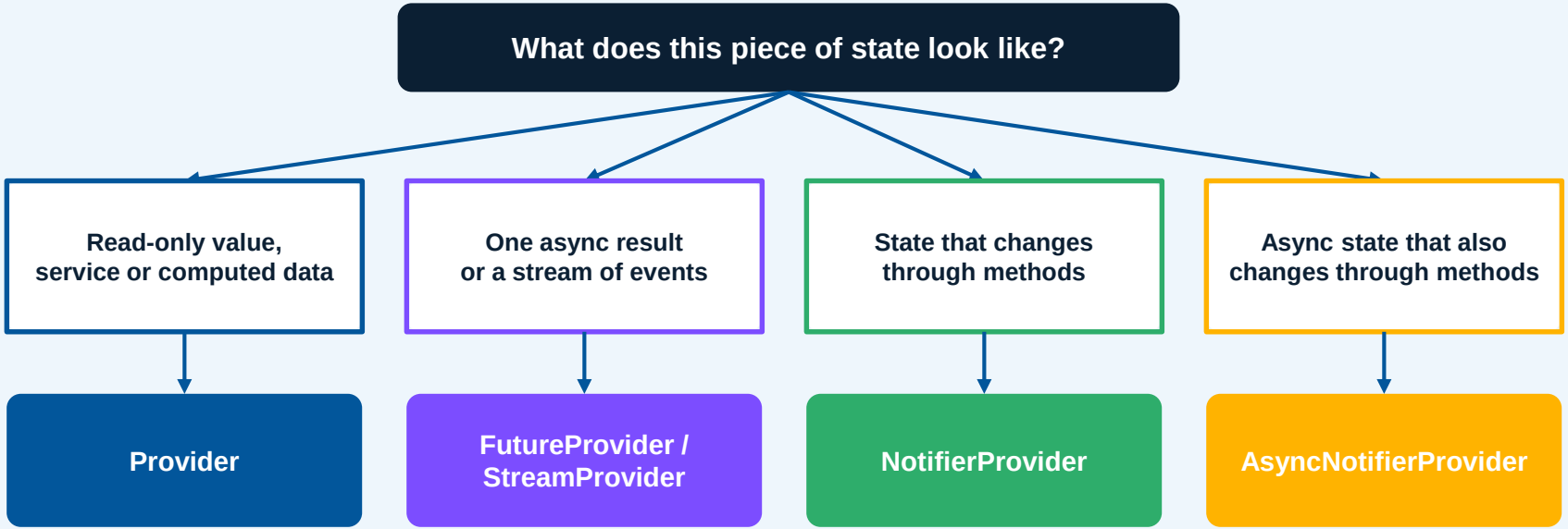
// DO: read in callbacks
onPressed: () {
  ref.read(counterProvider.notifier).increment();
}

# CI
dart analyze --fatal-infos
```

Sources: [1] What's new in 3.0, [21] pub.dev packages, [18] DO/DON'T

WHY IT MATTERS

Which provider do I need?



Every provider type answers one question: who owns this state and how does it change? The next slides explain each type and each Ref function with its purpose and an example.

Sources: [4] Providers, [14] AsyncValue API

Provider

PURPOSE

Exposes a read-only value that is created once and cached: a repository, an HTTP client, configuration, or data computed from other providers.

WHY IT MATTERS

It replaces singletons and service locators. Dependencies are declared in code, so tests can swap any of them, and derived values recompute only when their inputs change.

USE IT WHEN

The value never changes by itself, or is a pure function of other providers.

provider_example.dart

```dart
final apiBaseUrlProvider =
    Provider<String>((ref) => 'https://api.example.com');

final httpClientProvider = Provider<Dio>((ref) {
  final dio = Dio(BaseOptions(
      baseUrl: ref.watch(apiBaseUrlProvider)));
  ref.onDispose(dio.close); // cleanup
  return dio;
});

// Derived value: recomputed only when inputs change
final visibleCountProvider = Provider<int>((ref) {
  final todos = ref.watch(todoListProvider).value ?? [];
  return todos.where((t) => !t.done).length;
});

// Test: swap the real client for a fake
ProviderScope(overrides: [
  httpClientProvider.overrideWithValue(FakeDio()),
], child: const MyApp());
```

Sources: [4] Providers, [6] Refs

WHY IT MATTERS

FutureProvider

PURPOSE

Runs an async function and exposes the outcome as an AsyncValue, which is loading, data or error.

WHY IT MATTERS

You cannot forget the loading or error case, because the type forces you to handle them. Results are cached, shared by all listeners, and recomputed when a watched dependency changes.

USE IT WHEN

You load data once per input, such as a profile, a settings file or a search result.

future_provider.dart

```dart
final userProvider = FutureProvider<User>((ref) async {
  // Re-runs automatically when the token changes
  final token = ref.watch(authTokenProvider);
  return ref.watch(userRepositoryProvider).fetch(token);
});

class ProfileHeader extends ConsumerWidget {
  const ProfileHeader({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final user = ref.watch(userProvider);
    return switch (user) {
      AsyncData(:final value) => Text(value.name),
      AsyncError(:final error) => Text('Failed: $error'),
      _ => const CircularProgressIndicator(),
    };
  }
}
```

Sources: [4] Providers, [14] AsyncValue API

WHY IT MATTERS

StreamProvider

PURPOSE

Subscribes to a Stream and exposes the latest event as an AsyncValue.

WHY IT MATTERS

Riverpod handles subscribing, cancelling and re-subscribing for you, so there is no StreamBuilder boilerplate. Listeners share one subscription, and the stream is cancelled when nobody listens.

USE IT WHEN

You consume live data: chat messages, auth state, database or Firestore snapshots, sensors.

stream_provider.dart

```dart
final messagesProvider =
    StreamProvider.autoDispose<List<Message>>((ref) {
  // Switching chats cancels the old stream
  final chatId = ref.watch(selectedChatIdProvider);
  return ref.watch(chatRepositoryProvider).watch(chatId);
});

class MessageList extends ConsumerWidget {
  const MessageList({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return ref.watch(messagesProvider).when(
      data: (items) => ListView(children: [
        for (final m in items) ListTile(title: Text(m.text)),
      ]),
      error: (e, _) => Text('$e'),
      loading: () => const LinearProgressIndicator(),
    );
  }
}
```

Sources: [4] Providers, [2] Migration guide

WHY IT MATTERS

NotifierProvider

PURPOSE

Holds synchronous state together with the methods that are allowed to change it.

WHY IT MATTERS

The state can be changed only through the notifier's methods, so business rules live in one testable class instead of being spread through widgets. Widgets just call intent-revealing methods.

USE IT WHEN

State is changed by the user or events: a cart, filters, a form draft, theme or locale.

cart.dart

```dart
class Cart extends Notifier<List<Item>> {
  @override
  List<Item> build() => const []; // initial state

  void add(Item item) => state = [...state, item];

  void remove(String id) =>
      state = [for (final i in state) if (i.id != id) i];

  void clear() => state = const [];
}

final cartProvider =
    NotifierProvider<Cart, List<Item>>(Cart.new);

// Widget: read state with watch, call methods with read
final items = ref.watch(cartProvider);
onPressed: () =>
    ref.read(cartProvider.notifier).add(product),
```

Sources: [4] Providers, [1] What's new in 3.0

AsyncNotifierProvider

PURPOSE

Combines asynchronous loading in build() with methods that modify the data, usually by calling a server.

WHY IT MATTERS

It is the standard pattern for feature screens: loading and error states come for free, and every write goes through one class. AsyncValue.guard turns exceptions into an error state.

USE IT WHEN

A screen loads remote data and also creates, updates or deletes it.

todo_list.dart

```dart
class TodoList extends AsyncNotifier<List<Todo>> {
  @override
  Future<List<Todo>> build() =>
      ref.watch(todoRepositoryProvider).fetchAll();

  Future<void> add(String title) async {
    final repo = ref.read(todoRepositoryProvider);
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      await repo.create(title);
      return repo.fetchAll();
    });
  }
}

final todoListProvider =
    AsyncNotifierProvider<TodoList, List<Todo>>(
        TodoList.new);
```

Sources: [4] Providers, [14] AsyncValue API

WHY IT MATTERS

ref.watch

PURPOSE

Reads a provider and subscribes to it. When the value changes, the caller rebuilds.

WHY IT MATTERS

This is what makes the app reactive. Because dependencies are declared with watch, state flows downstream automatically and nothing needs to be refreshed by hand.

USE IT WHEN

Inside build() of a widget, or when creating a provider that depends on another one.

watch_example.dart

```dart
// In a provider: recompute when inputs change
final visibleTodosProvider = Provider<List<Todo>>((ref) {
  final todos = ref.watch(todoListProvider).value ?? [];
  final filter = ref.watch(filterProvider);
  return switch (filter) {
    Filter.all => todos,
    Filter.active => todos.where((t) => !t.done).toList(),
    Filter.done => todos.where((t) => t.done).toList(),
  };
});

// In a widget: rebuild when the list changes
Widget build(BuildContext context, WidgetRef ref) {
  final todos = ref.watch(visibleTodosProvider);
  return Text('${todos.length} todos');
}

// Never call watch inside onPressed or other callbacks
```

Sources: [6] Refs, [5] Consumers, [18] DO/DON'T

ref.read

PURPOSE

Reads the current value once, without subscribing. It never causes a rebuild.

WHY IT MATTERS

Callbacks run in response to events, not during a build. Using read there avoids needless subscriptions and the bugs that come from watching inside event handlers.

USE IT WHEN

Inside onPressed, onChanged, initState, or a notifier method that needs another provider.

read_example.dart

```dart
// In a callback: call a method on the notifier
FilledButton(
  onPressed: () =>
      ref.read(cartProvider.notifier).add(product),
  child: const Text('Add to cart'),
)

// In a notifier method: use a dependency once
Future<void> checkout() async {
  final api = ref.read(orderApiProvider);
  await api.submit(state);
  if (!ref.mounted) return; // 3.x safety check
  state = const [];
}

// Mistake: this value never updates on screen
Widget build(BuildContext context, WidgetRef ref) {
  final count = ref.read(counterProvider); // use watch
  return Text('$count');
}
```

Sources: [6] Refs, [18] DO/DON'T

WHY IT MATTERS

ref.listen

PURPOSE

Runs a callback when a provider changes, receiving the previous and next values, without rebuilding the widget.

WHY IT MATTERS

Some reactions are not UI: snackbars, dialogs, navigation and analytics. Putting them in build() would repeat them on every rebuild, while listen fires once per change.

USE IT WHEN

You need a side effect in response to state, such as showing an error or navigating after success.

listen_example.dart

```dart
class CheckoutPage extends ConsumerWidget {
  const CheckoutPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    ref.listen<AsyncValue<Receipt?>>(
      checkoutProvider,
      (previous, next) {
        if (next.hasError && !next.isLoading) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('${next.error}')),
          );
        }
        if (next.value != null) {
          context.go('/receipt');
        }
      },
    );
    return const CheckoutForm();
  }
}
```

Sources: [6] Refs, [5] Consumers

select

PURPOSE

Watches or listens to a single slice of a provider's state instead of the whole object.

WHY IT MATTERS

Large state objects change often. With select, a widget rebuilds only when its own field changes, which keeps lists and dashboards smooth without manual optimization.

USE IT WHEN

A widget needs one field, such as a user's name, an item count or a flag.

select_example.dart

```dart
// Rebuilds only when the name changes,
// not when other user fields change
final name = ref.watch(
  userProvider.select((user) => user.name),
);

// Cart badge: rebuild on count change only
final count = ref.watch(
  cartProvider.select((items) => items.length),
);

// Async providers: wait for one field
final avatar = await ref.watch(
  profileProvider.selectAsync((p) => p.avatarUrl),
);

// Listen to a slice as well
ref.listen(
  cartProvider.select((items) => items.isEmpty),
  (was, isEmpty) { if (isEmpty) showEmptyHint(); },
);
```

Sources: [17] Reduce rebuilds, [6] Refs

WHY IT MATTERS

family and autoDispose

PURPOSE

family creates one provider state per argument, such as per id. autoDispose frees state when nobody listens, and keepAlive keeps it.

WHY IT MATTERS

Without family you would hand-write a provider for each item. Without disposal, caches grow forever. Together they give screen-scoped, correctly cached state.

USE IT WHEN

Detail screens, search results and per-item data (family); temporary screen state (autoDispose).

family_dispose.dart

```dart
// One cached state per product id
final productProvider =
    FutureProvider.autoDispose.family<Product, String>(
        (ref, id) {
  // Keep the result 30s after the screen closes
  final link = ref.keepAlive();
  final timer = Timer(const Duration(seconds: 30), link.close);
  ref.onDispose(timer.cancel);
  return ref.watch(productRepositoryProvider).fetch(id);
});

ref.watch(productProvider('42')); // separate state
ref.watch(productProvider('7'));  // for each id

// Arguments need correct == and hashCode:
// use String, int, records or value classes
```

Sources: [8] Family, [7] Automatic disposal

Key takeaways

- Providers are a declarative dependency graph
- Notifiers and AsyncValue model state and async work
- 3.x adds retry, pause, Ref.mounted, mutations and persistence
- Code generation trims boilerplate; keep it optional
- Override providers to test everything

Latest at time of writing: flutter_riverpod 3.4.3 (Sep 2026)

References

[1] What's new in Riverpod 3.0
riverpod.dev/docs/whats_new

[2] Migrating from 2.0 to 3.0
riverpod.dev/docs/3.0_migration

[3] Getting started
riverpod.dev/docs/introduction/getting_started

[4] Providers
riverpod.dev/docs/concepts2/providers

[5] Consumers
riverpod.dev/docs/concepts2/consumers

[6] Refs
riverpod.dev/docs/concepts2/refs

[7] Automatic disposal
riverpod.dev/docs/concepts2/auto_dispose

[8] Family
riverpod.dev/docs/concepts2/family

[9] Mutations (experimental)
riverpod.dev/docs/concepts2/mutations

[10] Offline persistence (experimental)
riverpod.dev/docs/concepts2/offline

[11] Automatic retry
riverpod.dev/docs/concepts2/retry

[12] About code generation
riverpod.dev/docs/concepts/about_code_generation

[13] Testing your providers
riverpod.dev/docs/how_to/testing

[14] AsyncValue API reference
pub.dev/documentation/hooks_riverpod/latest/hooks_riverpod/AsyncValue-class.html

[15] ProviderObservers
riverpod.dev/docs/concepts2/observers

[16] Scoping providers
riverpod.dev/docs/concepts2/scoping

[17] How to reduce provider/widget rebuilds
riverpod.dev/docs/how_to/select

[18] DO / DON'T
riverpod.dev/docs/root/do_dont

[19] Riverpod for Provider users
riverpod.dev/docs/from_provider/quickstart

[20] Riverpod on GitHub
github.com/rrousselGit/riverpod

[21] Packages: flutter_riverpod, riverpod_generator, riverpod_lint, riverpod_sqflite
pub.dev/packages/flutter_riverpod

[22] ProviderContainers / ProviderScopes
riverpod.dev/docs/concepts2/containers